12-Month Career Master Plan

Learn → Implement → Prove → Apply → Improve

A weekly plan designed to make you broadly employable across administration, payroll, business/data roles and software engineering—while using your B.Tech CSE advantage.

How to use the plan

• **Daily rule:** learn a concept, then immediately implement it. Do not spend the whole day watching tutorials.

• **Weekly rule:** finish one tangible deliverable every week.

• **Proof rule:** save screenshots, GitHub commits, spreadsheets, documentation and project notes.

• **Job rule:** start applying once you have a usable skill set; do not wait until Month 12.

• **DSA rule:** from Week 17 onward, keep 45–60 minutes of DSA practice on most study days.

• **Project rule:** Driveway Docs becomes the main software project; upgrade it layer by layer rather than starting many unrelated projects.

Suggested daily schedule

Time	Focus	What you do
60–90 min	Learn	Study only the week's topic.
60–120 min	Implement	Build the weekly deliverable yourself.
30–60 min	Practice	Excel/SQL/DSA exercises depending on the phase.
15 min	Document	Write what you learned, bugs, decisions and next steps.
15–30 min	Career	Applications, resume tailoring, networking when job-search is active.

Important: You do not need to study every area every day. The schedule changes emphasis by phase. The goal is breadth plus one deep technical specialization.

Quarter 1 — Office, Admin, Payroll & Job Readiness

Week 1: Career setup + digital workplace

Learn: Set up VS Code, Git/GitHub, professional folders, cloud storage, LinkedIn/job tracker.

Implement: Create a career tracker; create GitHub; organize resume, certificates and projects.

Weekly proof: A clean professional workspace and application tracker.

Week 2: Word + professional documents

Learn: Word styles, headers, tables, page layout, PDFs, templates.

Implement: Create a business letter, memo, meeting agenda and minutes.

Weekly proof: Four polished office documents.

Week 3: Outlook + workplace communication

Learn: Email, calendar, scheduling, flags, folders, signatures, professional etiquette.

Implement: Create a mock weekly calendar; write 10 workplace emails.

Weekly proof: Can manage a basic office inbox/calendar.

Week 4: Excel fundamentals

Learn: Tables, formatting, sort/filter, freeze panes, basic formulas, data validation.

Implement: Build an employee/contact tracker and expense sheet.

Weekly proof: Accurate, clean Excel workbook.

Week 5: Excel formulas

Learn: IF, SUMIF/SUMIFS, COUNTIF/COUNTIFS, AVERAGEIF, text/date functions.

Implement: Build attendance and leave tracker with automatic summaries.

Weekly proof: Formulas work without manual calculation.

Week 6: Excel lookup + data cleaning

Learn: XLOOKUP, INDEX/MATCH concepts, duplicate removal, cleaning text/dates.

Implement: Build an employee master sheet pulling department/pay information from tables.

Weekly proof: Can combine multiple business datasets.

Week 7: Pivot Tables + dashboards

Learn: Pivot Tables, charts, slicers, basic dashboard design.

Implement: Build an office operations dashboard.

Weekly proof: One-page management dashboard.

Week 8: Admin operations

Learn: Data entry, records, filing, scheduling, invoices, purchase orders, confidentiality.

Implement: Create a mock office workflow from incoming email to completed record.

Weekly proof: Can explain an end-to-end admin process.

Week 9: Customer service

Learn: Professional phone/email handling, complaints, escalation, follow-up.

Implement: Create customer-service scenarios and response templates.

Weekly proof: Confident professional communication.

Week 10: Payroll foundations

Learn: Gross/net pay, CPP, EI, income tax, vacation pay, overtime, deductions.

Implement: Build a simple payroll practice workbook using fictional data; do not use real employee data.

Weekly proof: Understand Canadian payroll workflow at a basic level.

Week 11: Payroll administration

Learn: Payroll records, pay periods, reconciliation, T4/ROE concepts, statutory records.

Implement: Create a payroll checklist and reconciliation sheet using fictional data.

Weekly proof: Can support a payroll administrator.

Week 12: QuickBooks + bookkeeping basics

Learn: Invoices, bills, expenses, payments, AP/AR, bank reconciliation, financial reports.

Implement: Create a fictional small-business bookkeeping exercise.

Weekly proof: Understand basic bookkeeping workflow.

Week 13: Admin portfolio + resume

Learn: Resume tailoring, ATS basics, interview stories.

Implement: Package 3 Excel projects + office documents into a portfolio; tailor admin resume.

Weekly proof: Admin-ready application package.

Week 14: Job applications + interview prep

Learn: Job descriptions, keyword matching, STAR answers.

Implement: Apply to targeted admin/office/payroll-support roles; practice 5 interview questions.

Weekly proof: Begin consistent applications.

Quarter 2 — Python, DSA, SQL & Backend Foundations

Week 15: Python foundations

Learn: Variables, types, conditionals, loops, functions, lists, dicts, sets.

Implement: Write small scripts for expense totals, attendance summaries and data validation.

Weekly proof: Can write basic Python independently.

Week 16: Python OOP + clean code

Learn: Classes, methods, modules, exceptions, type hints.

Implement: Build a small employee/booking business-logic module.

Weekly proof: Can structure a small Python program.

Week 17: DSA: arrays + strings

Learn: Big-O basics, arrays, strings, common patterns.

Implement: Solve 8-10 beginner problems; implement reusable helpers.

Weekly proof: Understand basic complexity and patterns.

Week 18: DSA: hashing + two pointers

Learn: Hash maps/sets, frequency counting, two pointers.

Implement: Solve 8-10 problems; write explanations for each pattern.

Weekly proof: Can recognize common interview patterns.

Week 19: SQL fundamentals

Learn: SELECT, WHERE, ORDER BY, GROUP BY, aggregates.

Implement: Create a small business database and write 25 queries.

Weekly proof: Can retrieve business data confidently.

Week 20: SQL joins + subqueries

Learn: INNER/LEFT joins, subqueries, CTE basics.

Implement: Analyze customers, bookings, sales and employees across tables.

Weekly proof: Can combine relational datasets.

Week 21: SQL advanced + database design

Learn: Indexes, constraints, transactions, normalization, window functions.

Implement: Design Driveway Docs schema and ER diagram.

Weekly proof: Database design is deliberate and explainable.

Week 22: Django fundamentals

Learn: Project/app structure, models, ORM, URLs, views, migrations, admin.

Implement: Create Driveway Docs backend with accounts, services and bookings apps.

Weekly proof: Working Django MVP.

Week 23: Authentication + permissions

Learn: Login, registration, sessions/tokens, roles, permissions.

Implement: Implement customer/admin access controls.

Weekly proof: Users can only access permitted data.

Week 24: Booking engine

Learn: Availability, validation, status workflow, cancellation/rescheduling.

Implement: Implement Requested → Confirmed → Scheduled → In Progress → Completed.

Weekly proof: Core booking workflow works.

Week 25: REST APIs

Learn: Django REST Framework, serializers, ViewSets, routers, API validation.

Implement: Build auth, services and bookings APIs.

Weekly proof: Backend can operate through documented APIs.

Week 26: React fundamentals

Learn: Components, props, state, hooks, forms, routing.

Implement: Build customer-facing React UI.

Weekly proof: Frontend consumes API data.

Week 27: React dashboards

Learn: Admin/customer dashboards, loading/error states, reusable components.

Implement: Build booking history, admin bookings and service management screens.

Weekly proof: Usable full-stack application.

Week 28: Testing

Learn: pytest, Django tests, API tests, unit vs integration testing.

Implement: Test auth, permissions, booking conflicts, cancellations and APIs.

Weekly proof: Critical business rules have automated tests.

Quarter 3 — Full-Stack Engineering, Testing, Docker & Cloud

Week 29: PostgreSQL production use

Learn: PostgreSQL setup, migrations, indexes, transactions, query performance.

Implement: Move app to PostgreSQL; optimize slow queries.

Weekly proof: Reliable relational data layer.

Week 30: Redis + caching

Learn: Redis concepts, cache keys, expiry, invalidation.

Implement: Cache suitable read-heavy data such as service listings.

Weekly proof: Can explain when caching helps.

Week 31: Celery + background jobs

Learn: Queues, workers, retries, idempotency.

Implement: Send booking confirmations and reminders asynchronously.

Weekly proof: Background work does not block requests.

Week 32: Payment sandbox + webhooks

Learn: Payment states, webhooks, idempotency, failure handling.

Implement: Implement test/sandbox payment flow; store transaction metadata only.

Weekly proof: Payment workflow is safe and testable.

Week 33: Docker

Learn: Images, containers, Dockerfile, Compose, volumes, networks.

Implement: Containerize frontend, backend, PostgreSQL, Redis and worker.

Weekly proof: Full local stack starts consistently.

Week 34: Linux + deployment fundamentals

Learn: Processes, ports, environment variables, SSH, logs, basic networking.

Implement: Run the containerized app on a Linux environment.

Weekly proof: Comfortable troubleshooting a server.

Week 35: AWS foundations

Learn: IAM, EC2, RDS, S3, CloudWatch concepts.

Implement: Create a small AWS sandbox and document resources.

Weekly proof: Understands core cloud building blocks.

Week 36: Production AWS deployment

Learn: Secure config, HTTPS, RDS, S3, backups.

Implement: Deploy Driveway Docs to AWS using appropriate managed services.

Weekly proof: Application is accessible in a production environment.

Week 37: CI/CD

Learn: GitHub Actions, test/build/deploy pipeline.

Implement: Run tests on pull requests and deploy approved changes.

Weekly proof: Automated delivery pipeline.

Week 38: Security hardening

Learn: Secrets, authentication security, CSRF/CORS, XSS, injection, rate limiting.

Implement: Perform security checklist; create SECURITY.md.

Weekly proof: Security decisions are documented and implemented.

Week 39: Logging + monitoring

Learn: Structured logs, health checks, metrics, incident basics.

Implement: Add /health endpoint, useful logs and failure monitoring.

Weekly proof: Can diagnose common production failures.

Week 40: ETL fundamentals

Learn: Extract/transform/load, Pandas, data quality, idempotency.

Implement: Extract booking data, clean it and load analytics tables.

Weekly proof: Repeatable ETL pipeline.

Week 41: Analytics + Power BI

Learn: Power Query, data models, DAX basics, KPI design.

Implement: Create a Driveway Docs operations/revenue dashboard from analytics tables.

Weekly proof: Business + technical analytics capability.

Week 42: ETL orchestration

Learn: Scheduling concepts; Airflow basics.

Implement: Schedule a daily analytics pipeline and handle failed runs.

Weekly proof: Understands production data workflows.

Quarter 4 — Data, ETL, System Design, Portfolio & Interviews

Week 43: Data integration + APIs

Learn: External APIs, pagination, retries, rate limits, data mapping.

Implement: Integrate a safe public/test API into the project.

Weekly proof: Can design an integration workflow.

Week 44: System design basics

Learn: Scalability, load, caching, queues, database bottlenecks, failure modes.

Implement: Draw architecture and sequence diagrams for booking/payment/notification flows.

Weekly proof: Can explain system trade-offs.

Week 45: Reliability + incident response

Learn: Backups, recovery, root-cause analysis, postmortems.

Implement: Simulate DB outage, failed email job and bad deployment; document recovery.

Weekly proof: Can think beyond 'the code works'.

Week 46: Performance

Learn: Profiling, query optimization, caching, pagination, load concepts.

Implement: Measure API response time and optimize one bottleneck.

Weekly proof: Can explain a measurable performance improvement.

Week 47: Enterprise engineering habits

Learn: Code review, branching strategy, documentation, change management.

Implement: Open pull requests for your own project; write review comments and change notes.

Weekly proof: Demonstrates professional engineering workflow.

Week 48: Portfolio polish

Learn: README, architecture, API, database, security, deployment docs.

Implement: Create a polished project case study with screenshots and diagrams.

Weekly proof: Recruiter can understand project quickly.

Week 49: Resume for software roles

Learn: ATS, impact bullets, technology selection, project metrics.

Implement: Create junior software, backend and full-stack resume versions.

Weekly proof: Three targeted resume variants.

Week 50: Interview DSA + SQL

Learn: Arrays, strings, hashing, two pointers, SQL joins/window functions.

Implement: Solve timed problems and explain solutions aloud.

Weekly proof: Interview-ready fundamentals.

Week 51: Technical + behavioral interviews

Learn: Project walkthrough, STAR stories, debugging, system design basics.

Implement: Practice mock interviews; record 2-minute project explanation.

Weekly proof: Can confidently explain your work.

Week 52: Career launch + review

Learn: Review skills, gaps, applications, interviews and next specialization.

Implement: Apply broadly; analyze response rates; choose next 6-month specialization.

Weekly proof: A complete employability system and clear next step.

Production-Ready Driveway Docs — Final Checklist

■ Customer registration/login and role-based access work correctly.

■ Customers can view services, book, reschedule and cancel appointments.

■ Booking conflicts are prevented using application validation plus database integrity controls.

■ REST APIs are documented and protected.

■ PostgreSQL is used as the production relational database.

■ Important business workflows have automated tests.

■ Redis/Celery handle appropriate asynchronous work.

■ Payment integration uses a sandbox/test environment and handles failures safely.

■ Docker runs the application stack consistently.

■ AWS deployment is documented and uses secure configuration.

■ CI/CD automatically tests changes before deployment.

■ Security review and SECURITY.md are complete.

■ Health checks, structured logs and monitoring support troubleshooting.

■ ETL creates validated analytics data.

■ Power BI or an equivalent dashboard communicates business KPIs.

■ Architecture, database, API, deployment and testing documentation are complete.

■ You can explain the project, technical decisions, trade-offs, failures and improvements in an interview.

Career outcome to aim for

Months 1–3: Admin/office/payroll-support readiness + applications.

Months 4–6: Strong Python/SQL/Django foundation + junior tech applications.

Months 7–9: Full-stack, testing, Docker and cloud + stronger software applications.

Months 10–12: ETL, analytics, system design, production operations and interview readiness.

The objective is not to become an expert in every career. It is to become broadly employable while developing one strong technical specialization that can grow into a long-term software/data career.